UI tool kit, you install another package. For example, to use Bootstrap 5, run `python -m pip install crispy-bootstrap5`, add `"crispy_bootstrap5"` to `INSTALLED_APPS`, and add two new configuration values: `CRISPY_ALLOWED_TEMPLATE_PACKS = "bootstrap5"` and `CRISPY_TEMPLATE_PACK = "bootstrap5"`.

There are two ways to use Crispy Forms in your Django templates. The first is the `| crispy` filter. This filter gets applied to a form object inside your template, so instead of using `form.as_p` to output the form, you use `form | crispy`. This filter outputs a canned layout and style based on the installed template pack. There is no extra control over the look; you can't customize it at all. Consider it an opinionated layout, like `.as_p`, but with your template pack add-on.

The second way to use Crispy Forms is with the `{% crispy %}` tag. This is more complicated than the filter but more powerful as well. Within the template, you use the tag, giving the form object as an argument. To fully take advantage of the tag, you add Crispy Form's `FormHelper` class to your `ModelForm` class. Between the helper and an associated `Layout` object you get fine-grained control over the form's HTML, including tag IDs, CSS classes, the form action, and the HTTP method. Specific template packs also have their own tool-kit-specific attributes, which you can modify. For example, when using the Bootstrap pack, you can change whether error messages get rendered inline or as a block by setting `.error_text_inline=False`. As all this customization is done in your Python code, rather than in your templates, you can programmatically control the form inside a view. Crispy Forms provides as much or as little control as you want, with helpful defaults to make things prettier with little effort.

As mentioned in chapter 7, Django 5 has added a feature to make it easier to override and control the templates associated with a form field. This feature makes it easier to do the things Crispy Forms does for you, but it still means you have to write the underlying template widgets. Unless you're using a UI tool kit that isn't supported by a Crispy Forms template pack, you're better off using Crispy Forms, as someone else has done all the work of creating those templates for you.

13.3.3 Favicons

The little icon that appears on your browser's tab next to the page title is known as a *favicon*. This is another one of those nonstandard standards. It came about as a feature in Microsoft's Internet Explorer 5, which checked for the existence of a file named favicon.ico in the root directory of a website. Eventually, enough browsers adopted this practice that it became a World Wide Web Consortium (W3C) standard. As part of the standard, you can use a `<link>` tag in your HTML document's head to specify where to look for the image and which image format gets used. Unfortunately, adoption of the standard was a bit spotty, and different browsers support different image types, sizes, and variations of the attributes in that `<link>` tag.

Favicons are images, so from a Django perspective, they should be stored in your static folder. Django serves static files under the URL directory you configured with `STATIC_URL`, which is typically /static/ (see chapter 7 for more info). You can write